Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 3

The Command Line: Shell, Pipes and Text Filters

Track A: The Real OS, from the Outside (Linux)

Elaborated by:
st. gr. FAF-243
Veacheslav Morari

Verified by:
Patricia Reitman

Chișinău – 2026

Contents

Introduction **2**

1. Move fast, find help **2**

1.1. Observations 3

2. What the shell does to your line **3**

2.1. Observations 5

3. Redirection, chaining and jobs **5**

3.1. Observations 6

4. Answer questions with one-liners **6**

5. Your first script **9**

5.1. Observations 10

Conclusion **11**

Introduction

The purpose of this work is to learn to move quickly in bash, to predict what the shell does with a command line before the command runs, to redirect input and output, and to answer questions about data with one line of small filters joined by pipes. The last step is to turn the best one-liners into a first shell script.

1. Move fast, find help

I used Tab completion for paths: `ls /usr/sh` followed by Tab completed to `/usr/share/`, and `cd /et` followed by one Tab completed to `cd /etc/`. Tab asks the shell to finish the word under the cursor; when several names match, a second Tab prints the list of candidates. Ctrl+R opens a reverse search through the command history: after typing a few letters of an older command, the shell shows the most recent command containing them, and Enter runs it. In my run the command I had typed earlier appeared in the search line immediately after the first letters, and Enter executed it without retyping.

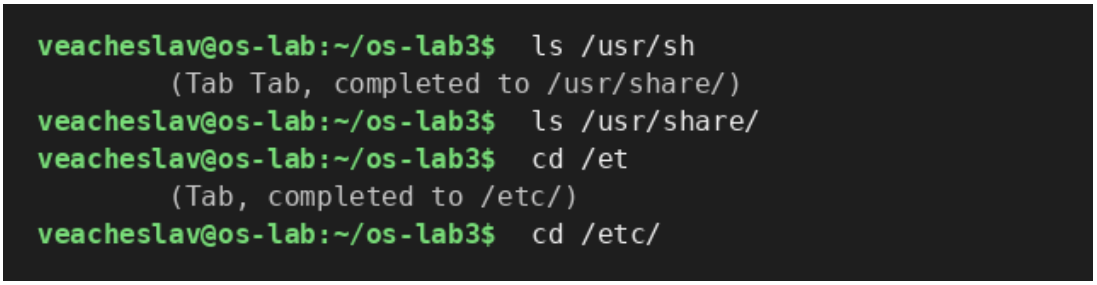


Figure 1.1 – Tab completion

The commands `type cd ; type ls ; type echo ; which ls` gave the following output.

```
cd is a shell builtin
ls is aliased to `ls --color=auto'
echo is a shell builtin
/usr/bin/ls
```

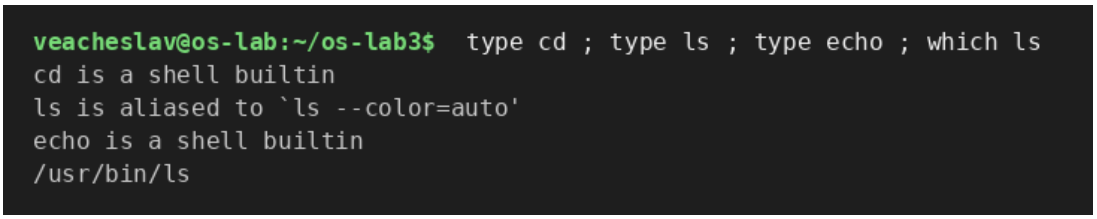


Figure 1.2 – Output of type and which

`echo $PATH | tr ':' '\n'` printed ten lines: `/usr/local/sbin`, `/usr/local/bin`, `/usr/sbin`, `/usr/bin`, `/sbin`, `/bin`, `/usr/games`, `/usr/local/games` and `/snap/bin` twice.

```
veacheslav@os-lab:~/os-lab3$  echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/sn
    ap/bin:/snap/bin
veacheslav@os-lab:~/os-lab3$  echo $PATH | tr ':' '\n'
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin
/snap/bin
```

Figure 1.3 – The directories in PATH

The command `nosuchprog ; echo "exit code: $?"` printed `nosuchprog: command not found` and `exit code: 127`.

```
veacheslav@os-lab:~/os-lab3$  nosuchprog ; echo "exit code: $?"
nosuchprog: command not found
exit code: 127
```

Figure 1.4 – Exit code of a missing command

Help was found with `man ls`, `ls --help | head`, `man -k "list directory"` and `help cd`.

1.1. Observations

(1) `cd` and `echo` are built-ins; `ls` is an external program (`/usr/bin/ls`), and in my shell it is also wrapped by the alias `ls --color=auto`. `cd` must be a built-in because every external command runs in a child process, and a child cannot change the current directory of its parent. If `cd` were a separate program, it would change its own directory and exit, and the shell would stay where it was. (2) `PATH` has ten entries, but `/snap/bin` is listed twice, so there are nine different directories. `ls` is in `/usr/bin`. (3) A command that is not found returns exit code 127. (4) The option `-t` sorts by modification time, newest first.

2. What the shell does to your line

In the test directory `~/os-lab3/play` I created the files `a.c b.c file1.txt file2.txt notes.txt` and `alfa beta.txt`.

```
veacheslav@os-lab:~/os-lab3$  mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
veacheslav@os-lab:~/os-lab3/play$  touch a.c b.c file1.txt file2.txt notes.txt "alfa
    beta.txt"
veacheslav@os-lab:~/os-lab3/play$  ls
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
```

Figure 2.1 – Files created in the play directory

Command	My prediction	Real output	Explanation
`echo *.c`	`a.c b.c`	`a.c b.c`	The shell expands the glob before `echo` starts.
`echo file?.txt`	`file1.txt file2.txt`	`file1.txt file2.txt`	`?` matches exactly one character.
`echo [a-b]*`	`a.c alfa beta.txt b.c`	`a.c alfa beta.txt b.c`	Any name starting with a or b; the name with a space stays one argument.
`echo *.xyz`	`*.xyz`	`*.xyz`	No file matches, so bash leaves the pattern unchanged.
`ls alfa beta.txt`	two errors	two "No such file" errors	Word splitting made two arguments, `alfa` and `beta.txt`.
`ls "alfa beta.txt"`	`alfa beta.txt`	`'alfa beta.txt'`	Quotes keep it as one argument.
`echo '$HOME' "$HOME" \{}$HOME`	`$HOME /home/veacheslav $HOME`	`$HOME /home/veacheslav $HOME`	Single quotes and backslash stop expansion; double quotes allow it.
`echo {1..5} ; echo $((7 * 6))`	`1 2 3 4 5, 42`	`1 2 3 4 5 and 42`	Brace expansion and arithmetic expansion.
`echo "today is $(date +%A), I am $(whoami)"`	`today is Monday, I am veacheslav`	`today is Monday, I am veacheslav`	Command substitution runs the commands and inserts their output.
`touch log{1..3}.txt && ls log*`	`log1.txt log2.txt log3.txt`	`log1.txt log2.txt log3.txt`	Braces create three names; `ls` then sees the glob result.

```
veacheslav@os-lab:~/os-lab3/play$  echo *.c
a.c b.c
veacheslav@os-lab:~/os-lab3/play$  echo file?.txt
file1.txt file2.txt
veacheslav@os-lab:~/os-lab3/play$  echo [a-b]*
a.c alfa beta.txt b.c
veacheslav@os-lab:~/os-lab3/play$  echo *.xyz
*.xyz
```

Figure 2.2 – Globbing results

```
veacheslav@os-lab:~/os-lab3/play$  ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
veacheslav@os-lab:~/os-lab3/play$  ls "alfa beta.txt"
'alfa beta.txt'
veacheslav@os-lab:~/os-lab3/play$  ls alfa\ beta.txt
'alfa beta.txt'
```

Figure 2.3 – The ls error on a name with a space, and the fix

```
veacheslav@os-lab:~/os-lab3/play$  echo '$HOME' "$HOME" \$HOME
$HOME /home/veacheslav $HOME
veacheslav@os-lab:~/os-lab3/play$  echo {1..5} ; echo $((7 * 6))
1 2 3 4 5
42
veacheslav@os-lab:~/os-lab3/play$  echo "today is $(date +%A), I am $(whoami)"
today is Monday, I am veacheslav
veacheslav@os-lab:~/os-lab3/play$  touch log{1..3}.txt && ls log*
log1.txt  log2.txt  log3.txt
```

Figure 2.4 – Single quotes, double quotes and backslash

2.1. Observations

(2) The file names were produced by the shell, not by `echo`. Bash replaces the pattern `*.c` with the matching names before it starts the program, so `echo` only receives `a.c b.c` as ordinary arguments. (3) `ls alfa beta.txt` failed because the shell split the line at the space into two arguments, and neither `alfa` nor `beta.txt` exists. It can be fixed by quoting, `ls "alfa beta.txt"`, or by escaping the space, `ls alfa\ beta.txt`. (4) Inside single quotes everything is literal: no variable, command or glob is expanded. Inside double quotes the shell still expands `$variables`, `$(commands)` and `$((arithmetic))`, but it does not split the result into words or expand globs. `ls` shows `'alfa beta.txt'` in quotes only to make the name readable when it contains a space; the quote characters are not part of the real file name.

3. Redirection, chaining and jobs

The commands `ls /etc /nope > out.txt 2> err.txt` and `wc -l out.txt ; cat err.txt` printed `238 out.txt` and `ls: cannot access '/nope': No such file or directory`.

```
veacheslav@os-lab:~/os-lab3$  ls /etc /nope > out.txt 2> err.txt
veacheslav@os-lab:~/os-lab3$  wc -l out.txt ; cat err.txt
238 out.txt
ls: cannot access '/nope': No such file or directory
```

Figure 3.1 – stdout and stderr in separate files

I compared both orders of redirection. With `ls /etc /nope > all.txt 2>&1` the error text was found by `grep nope all.txt`. With `ls /etc /nope 2>&1 > all.txt` the error appeared on the screen and `grep nope all.txt` printed nothing.

```
veacheslav@os-lab:~/os-lab3$  ls /etc /nope > all.txt 2>&1
veacheslav@os-lab:~/os-lab3$  grep nope all.txt
ls: cannot access '/nope': No such file or directory
veacheslav@os-lab:~/os-lab3$  ls /etc /nope 2>&1 > all.txt
ls: cannot access '/nope': No such file or directory
veacheslav@os-lab:~/os-lab3$  grep nope all.txt
```

Figure 3.2 – Comparing the order of redirections

Other results: `echo first > notes.md ; echo second >> notes.md ; cat notes.md` printed `first` and `second`; `wc -l < access.log` printed `500`, while `wc -l access.log` printed `500 access.log`; and `ls /nope 2> /dev/null || echo "failed with code $?"` printed `failed with code 2`. The line `mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3` printed `I am in /home/veacheslav/os-`

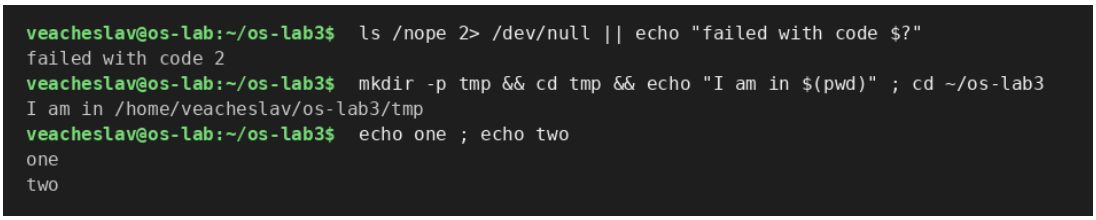


Figure 3.3 – The ; && || operators

The background job: `sleep 100 &` printed `[1] 9324`; the first `jobs` showed `Running`, and after `kill %1` the next `jobs` showed `Terminated`.

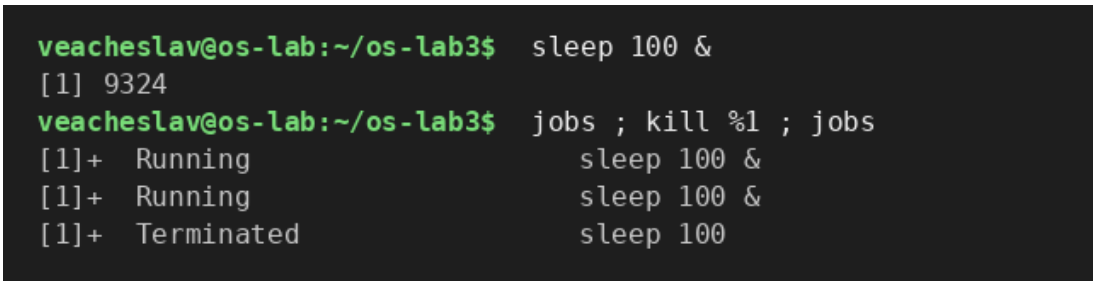


Figure 3.4 – Running a background job and stopping it

3.1. Observations

(1) The listing of `/etc` went to `out.txt` (standard output, descriptor 1) and the error about `/nope` went to `err.txt` (standard error, descriptor 2). (2) Redirections are processed from left to right. In `> all.txt 2>&1` stdout is first pointed at the file, and then stderr is made a copy of stdout, so both go to the file. In `2>&1 > all.txt` stderr is first made a copy of stdout while stdout still points at the terminal, and only afterwards stdout moves to the file; therefore the error stays on the screen and the file contains only the listing. (3) With `<` the shell opens the file and gives it to `wc` as standard input, so `wc` never learns the file name and prints only the number. With `wc -l access.log` the name is an argument, so `wc` prints it after the count. (4) The operator `;` runs the next command always, one after another. The operator `&&` runs the next command only if the previous one succeeded (exit code 0). The operator `||` runs the next command only if the previous one failed (non-zero exit code).

4. Answer questions with one-liners

Each line of `access.log` has the form date, time, user, result, page. The commands are saved in `oneliners.sh`; the results below come from `bash oneliners.sh` on the regenerated log of 500 lines.

Q1. Failed requests.

```
grep -c ' FAIL ' access.log
```

It printed `100`.

Q2. Different pages.

```
cut -d' ' -f5 access.log | sort -u | nl
```

It printed four pages: `/page0`, `/page1`, `/page2` and `/page3`.

Q3. Requests per page.

```
cut -d' ' -f5 access.log | sort | uniq -c
```

Each of the four pages got 125 requests; the sum is 500, as expected.

Q4. Users with the most failures.

```
grep ' FAIL ' access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk '
↪ NR==1{m=$1} $1==m'
```

It printed `15 user5` and `15 user3`; the two users are tied. All seven per-user counts (15, 15 and five times 14) add up to 100, which equals Q1.

Q5. Requests of user3.

```
echo "user3: $(grep -c ' user3 ' access.log) requests, $(grep ' user3 '
↪ access.log | grep -c ' FAIL ') failed"
```

It printed `user3: 72 requests, 15 failed`.

Q6. Last three failures.

```
grep ' FAIL ' access.log | tail -n 3 | cut -d' ' -f2,3
```

It printed `10:10 user0`, `10:15 user5` and `10:20 user3`.

Q7. Login shells.

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

The output lists each login shell with the number of accounts using it: `/sbin/nologin` (32), `/usr/sbin/nologin` (20), `/bin/bash` (4) and `/bin/sync` (1). The counts depend on the system.

Q8. ls versus ls -l.

```
echo "ls: $(ls /etc | wc -l), ls -l: $(ls -l /etc | wc -l), first line of ls
↪ -l: $(ls -l /etc | head -n 1)"
```

My prediction was that `ls -l` adds one extra line. The run confirmed it: the second count is larger by one (165 and 166 in the run shown in the figure). The difference is exactly one because `ls -l` prints an extra first line `total N` (the disk blocks used) before the lines of the files, and `wc -l` counts it as a line.

Bonus.

```
grep -E ' 10:[0-5]0 user[0-9] FAIL ' access.log | wc -l
```

It printed `50`. A request fails when its number is a multiple of 5, and the minute ends in 0 when the number is a multiple of 10; so every multiple of 10 up to 500 qualifies, which gives 50.

```
veacheslav@os-lab:~/os-lab3$  grep -c ' FAIL ' access.log
100
```

Figure 4.1 – Number of failed requests

```
veacheslav@os-lab:~/os-lab3$  cut -d' ' -f5 access.log | sort -u | nl
     1  /page0
     2  /page1
     3  /page2
     4  /page3
```

Figure 4.2 – Different pages

```
veacheslav@os-lab:~/os-lab3$  cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

Figure 4.3 – Requests per page

```
veacheslav@os-lab:~/os-lab3$  grep ' FAIL ' access.log | cut -d' ' -f3 | sort | uniq -c |
    sort -rn | awk 'NR==1{m=$1} $1==m'
     15 user5
     15 user3
```

Figure 4.4 – Users with the most failures

```
veacheslav@os-lab:~/os-lab3$  echo "user3: $(grep -c ' user3 ' access.log) requests, $(grep '
     user3 ' access.log | grep -c ' FAIL ') failed"
user3: 72 requests, 15 failed
```

Figure 4.5 – Requests of user3

```
veacheslav@os-lab:~/os-lab3$  grep ' FAIL ' access.log | tail -n 3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.6 – Last three failed requests

```
veacheslav@os-lab:~/os-lab3$  cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     32 /sbin/nologin
     20 /usr/sbin/nologin
      4 /bin/bash
      1 /bin/sync
```

Figure 4.7 – Login shells in /etc/passwd

```
veacheslav@os-lab:~/os-lab3$  echo "ls: $(ls /etc | wc -l), ls -l: $(ls -l /etc | wc -l),
    first line of ls -l: $(ls -l /etc | head -n 1)"
ls: 165, ls -l: 166, first line of ls -l: total 832
```

Figure 4.8 – ls compared with ls -l

```
veacheslav@os-lab:~/os-lab3$  bash oneliners.sh
100
     1  /page0
     2  /page1
     3  /page2
     4  /page3
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user5
     15 user3
user3: 72 requests, 15 failed
10:10 user0
10:15 user5
10:20 user3
     32 /sbin/nologin
     20 /usr/sbin/nologin
      4 /bin/bash
      1 /bin/sync
ls: 165, ls -l: 166, extra first line of ls -l: total 832
50
```

Figure 4.9 – Full run of oneliners.sh

5. Your first script

The script `report.sh` reads the log name from the first argument, stops with an error if the file does not exist, prints the totals and then loops over `user0` … `user6`.

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
        echo "no such file: $log" >&2
        exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for n in 0 1 2 3 4 5 6; do
        count=$(grep "user$n " "$log" | grep -c FAIL)
        echo "user$n: $count failures"
done
```

```
veacheslav@os-lab:~/os-lab3$  cat report.sh
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
        echo "no such file: $log" >&2
        exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for n in 0 1 2 3 4 5 6; do
        count=$(grep "user$n " "$log" | grep -c FAIL)
        echo "user$n: $count failures"
done
```

Figure 5.1 – Contents of report.sh

After `chmod +x report.sh`, `./report.sh access.log` printed `500 requests, 100 failed` followed by 14 failures for each user except `user3` and `user5`, which have 15. For a missing file, `./report.sh nope.log ; echo "exit code: $?"` printed `no such file: nope.log` and `exit code: 1`.

```
veacheslav@os-lab:~/os-lab3$  chmod +x report.sh && ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 5.2 – Running the script on access.log

```
veacheslav@os-lab:~/os-lab3$  ./report.sh nope.log ; echo "exit code: $?"
exit code: 1
no such file: nope.log
```

Figure 5.3 – Error handling for a missing file

5.1. Observations

(1) The first line `#!/bin/bash` (the shebang) tells the kernel which interpreter must run the file. Without `chmod +x` the file has no execute permission, so `./report.sh` fails with `Permission denied`. (2) The current directory is not in `PATH`, so the shell does not look for programs there; `./` gives an explicit path to the file. Without it, the shell would search only the `PATH` directories and answer `command not found`. (3) The message goes to standard error with `>&2` so that it is not mixed with the normal output when that output is redirected or piped. The command `exit 1` returns a non-zero code, so that other commands, `&&` and `||` chains or scripts can see that the script failed.

Conclusion

In this work I practised the everyday tools of the command line. I saw that the shell, not the program, expands globs, braces, variables and command substitutions, and that quoting decides which of these happen. I learned that redirections are applied from left to right, which is why `2>&1 > file` and `> file 2>&1` behave differently, and that exit codes (127 for a missing command, 0 for success) are what `&&` and `||` react to. Small filters such as `grep`, `cut`, `sort`, `uniq` and `wc` joined by pipes answered all questions about the log in one line each, and the control sums (500 requests and 100 failures) confirmed the results. Finally, I put these tools into a script with an argument check, a loop and a proper error exit code.